

AI WEATHERWISE

Phase 6: Project Testing

API test cases, results, evidence and observations

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	6 of 8 – Project Testing
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Objectives

The testing phase verifies that the AI WeatherWise APIs behave as specified: correct status codes, correct response bodies, secure authentication and acceptable response times. Tests were run with **Postman** against the locally running server.

2. Test Environment

Server	Node.js / Express running locally (<code>npm start</code>)
Base URL	<code>http://localhost:5001/api</code>
Database	MongoDB (local)
Test tool	Postman (Thunder Client can also be used)
Test data	name: John Doe, email: john@mail.com, password: 123456, city: Bangalore
Mode	Weather in fallback mode (<code>isMock: true</code>)

3. Test Strategy

- **Functional API testing:** each endpoint is called with valid data and the response is verified.
- **Negative testing:** invalid or missing data must return a clear error.
- **Security testing:** protected routes must reject requests without a valid JWT.
- **Performance observation:** the response time shown by Postman is recorded.

4. Test Cases and Results

4.1 Executed test cases (with screenshots)

ID	Test case	Input	Expected result	Actual result	Result
TC-01	Register a new user	POST /api/auth/register { name, email, password }	201 Created; user data and token	201 Created (172 ms); <code>_id</code> , name, email, token returned	Pass
TC-02	Login with valid credentials	POST /api/auth/login { email, password }	200 OK; user data and token	200 OK (90 ms); token returned	Pass
TC-03	Get weather for a city	GET /api/weather/bangalore	200 OK; weather metrics	200 OK (305 ms); temperature 34, humidity 79, windSpeed 12, condition Windy, isMock true	Pass
TC-04	Get AI weather recommendation	POST /api/ai/weather-recommendation { city, temperature, humidity, condition }	200 OK; recommendation text	200 OK (475 ms); "Stay hydrated, and wear light cotton clothes."	Pass

4.2 Additional test cases planned

These cases extend coverage and should be executed before the final release.

ID	Test case	Expected result
TC-05	Register with an already used e-mail	Error response (duplicate e-mail); no new user created
TC-06	Register with password shorter than 6 characters	Validation error message
TC-07	Register with an invalid e-mail format	Validation error message
TC-08	Login with wrong password	Authentication error; no token returned
TC-09	Call a protected route without a token	401 Unauthorized
TC-10	Call a protected route with an invalid token	401 Unauthorized
TC-11	Add the same city and country twice for one user	Duplicate rejected by the unique index
TC-12	Send NoSQL operators in the request body	Request sanitized or rejected

5. Test Evidence

5.1 TC-01 Register API

A POST request to `/api/auth/register` with name, email and password returned **201 Created** in 172 ms with the new user's `_id`, name, email and a JWT token.

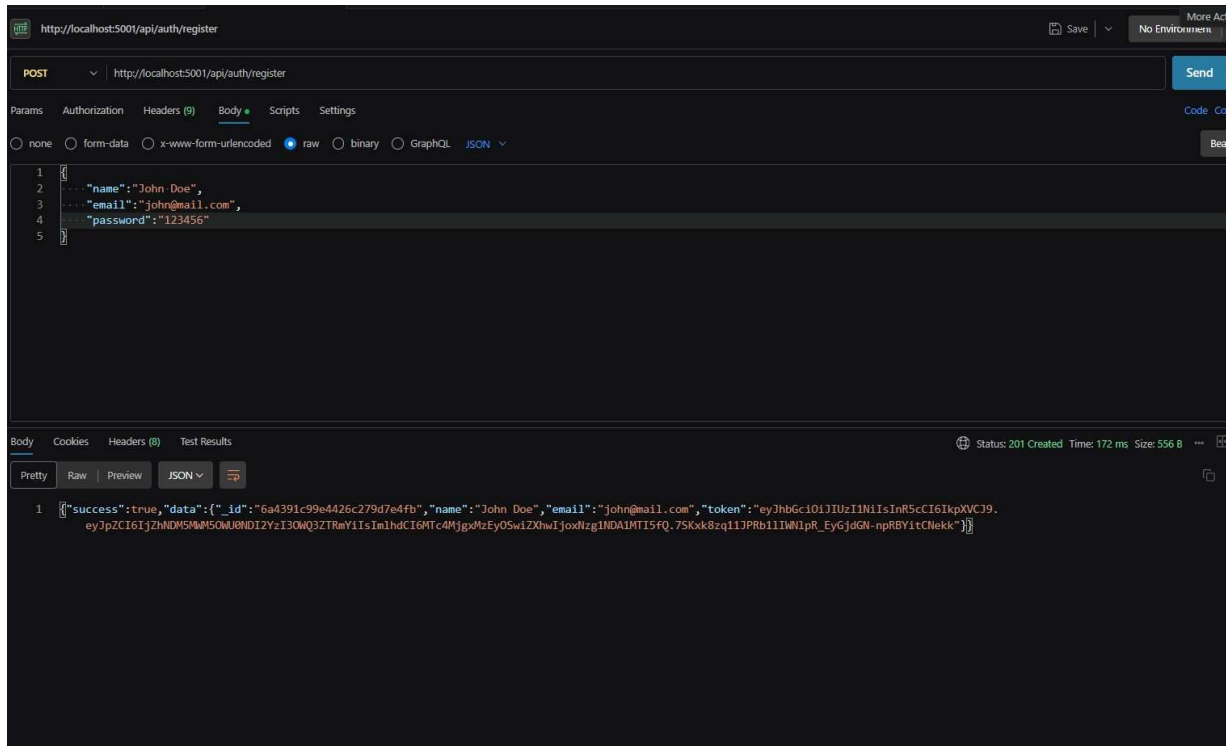


Figure 1: Postman – Register API (201 Created)

5.2 TC-02 Login API

A POST request to `/api/auth/login` with email and password returned **200 OK** in 90 ms with the user data and a fresh token.

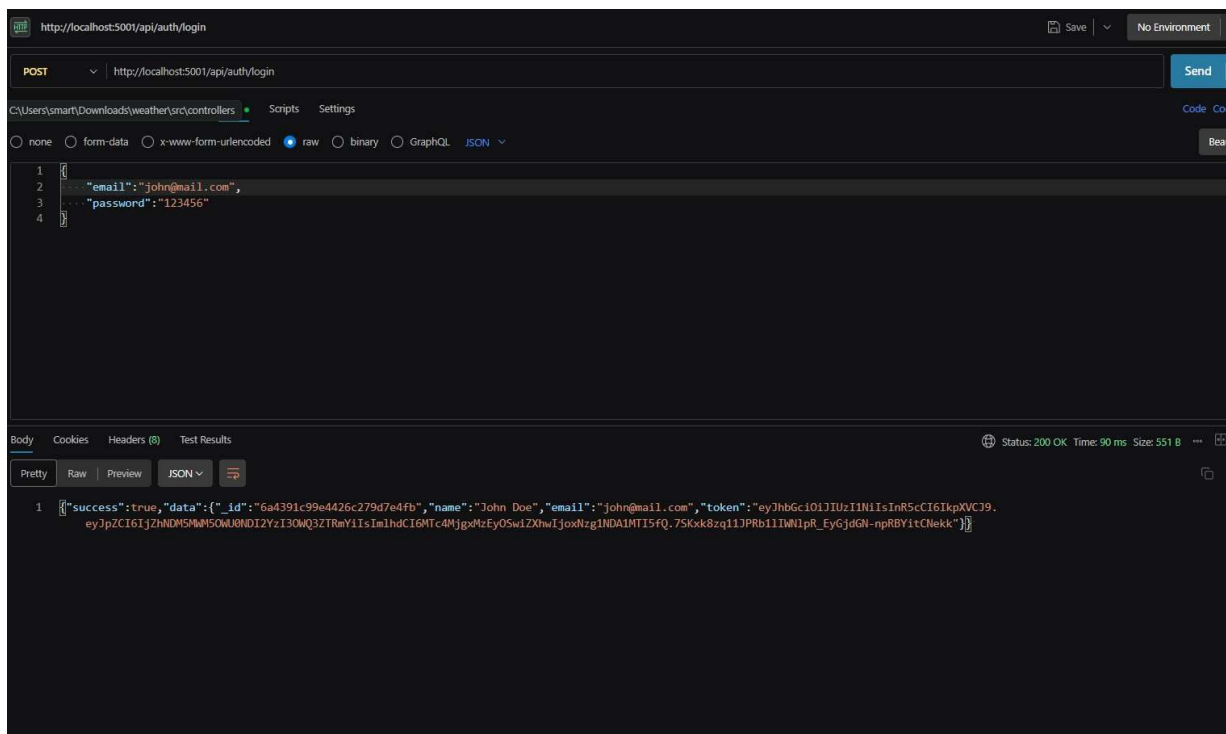


Figure 2: Postman – Login API (200 OK)

5.3 TC-03 Weather API

A GET request to `/api/weather/bangalore` returned **200 OK** in 305 ms. The `isMock: true` flag confirms the fallback mode is working when no live weather key is configured.

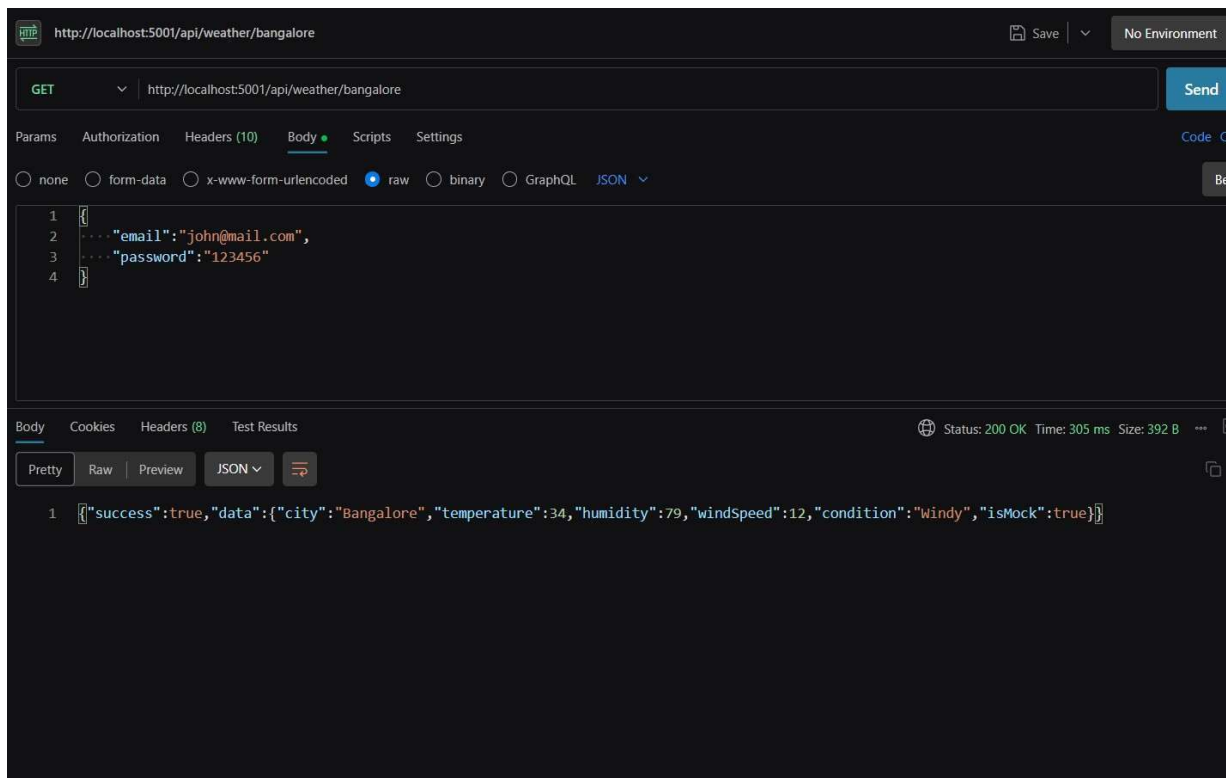


Figure 3: Postman – Weather API (200 OK)

```
{ "success": true, "data": { "city": "Bangalore", "temperature": 34, "humidity": 79, "windSpeed": 12, "condition": "Windy", "isMock": true } }
```

5.4 TC-04 AI Recommendation API

A POST request to `/api/ai/weather-recommendation` with city, temperature, humidity and condition returned **200 OK** in 475 ms with an actionable recommendation.

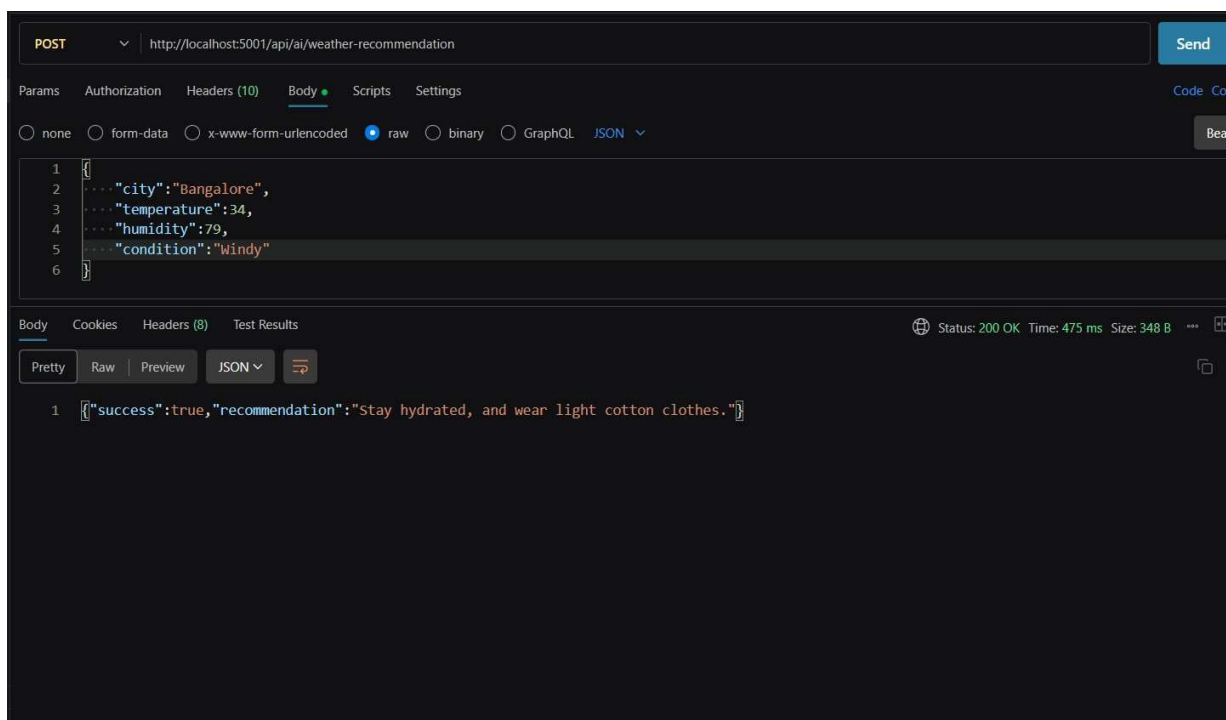


Figure 4: Postman – AI Recommendation API (200 OK)

6. Response Time Summary

API	Method	Status	Time	Size
/api/auth/register	POST	201 Created	172 ms	556 B
/api/auth/login	POST	200 OK	90 ms	551 B
/api/weather/bangalore	GET	200 OK	305 ms	392 B
/api/ai/weather-recommendation	POST	200 OK	475 ms	348 B

7. Observations and Defects

#	Observation	Suggested action
O1	The weather response is mock data (<code>isMock: true</code>) because a live weather key was not used in the test run.	Repeat TC-03 with a valid OpenWeatherMap key before the final demo.
O2	The GET weather request in Postman contains a JSON body (e-mail and password). A GET request does not need a body.	Remove the body from the request and save the corrected Postman collection.
O3	Testing used port 5001 while the code default is 5000.	Document the <code>PORT</code> value in the README.
O4	Negative and security test cases (TC-05 to TC-12) are not yet executed.	Run them and add screenshots.

8. Test Summary

Total executed	Passed	Failed	Planned (pending)
4	4	0	8

All executed functional tests passed with fast response times. Once the planned negative and security tests are completed, the API can be declared ready for demonstration. The test videos referenced in the project are: **API testing.mp4** and **Backend demo.mp4**.